

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2025/2026

UECS3253 WIRELESS APPLICATION DEVELOPMENT**Personal Finance Tracker**

Group Name:			
Student Name	Student ID	Practical Group	Programme
Chin Lok Bin	2400968	P4	SE
Kiu Chun Woon	2401624	P4	SE
Tan Yi Wen	2400800	P4	SE
Wong Xiang Rou	2401634	P4	SE

PART		CO	Component	Criteria	Marks Allocated
A	1	CO2	Report	Screens & Layout	/5
	2			Use of Appropriate Components	/5
	3			Reusability (Custom Components) inserts/updates	/5
	4			Implementation of React-Navigation: Stack, Drawer, Tab	/10
	5		Demonstration	User Interface	/5
Subtotal					/30
B	1	CO3	Report	Appropriate Usage of Data Persistence	/10
	2			Implementation of Data Persistence	/10
	3			CRUD Operations & input validation	/10
	4		Demonstration	CRUD and Data Persistence	/5
Subtotal					/35
C	1	CO4	Report	Appropriate Usage of Cloud Connectivity	/15
	2			Implementation of Cloud Connectivity	/15
	3		Demonstration	Cloud Connectivity	/5
Subtotal					/35

[Total: /100 marks]

Lee Kong Chian Faculty of Engineering and Science
Universiti Tunku Abdul Rahman

TABLE OF CONTENTS

PART

A	SCREENS, COMPONENTS AND REACT-NAVIGATION	4
	A.1 Screens & Layout	4
	A.2 Use of Components	5
B	DATA PERSISTENCE	12
	B.1 Usage of Data Persistence	12
	B.2 Implementation of Data Persistence	13
	B.3 CRUD Operations & Input Validation	21
	B.3.1 Create	21
	B.3.2 Read	23
	B.3.3 Update	25
	The update operation follow the same delayed, offline side architecture as the create and delete operation to guarantee the strong user experience accross changes of network conditions.	25
	B.3.4 Delete	29
	CLOUD CONNECTIVITY	31
	C.1 Usage of Cloud Connectivity	31
	C.1.1 REST API Data Synchronization:	32
	C.1.2 Real-Time WebSocket Communication	34
	C.1.3 External Currency System Integration	36
	C.2 Implementation of Cloud Connectivity	37
	C.2.1 REST API Implementation	37
	C.2.2 WebSocket Implementation	43
	C.2.3 Currency Exchange Rate Integration	43
	REFERENCES	45

BRIEF SUMMARY

The Personal Financial Tracker is a comprehensive, cross-platform mobile application developed using React Native. Designed with a modern, minimalist and aesthetic style, this application aims to empower users to seamlessly track, manage, and analyse their daily income and expenses. Structurally, the application utilizes a sophisticated 3-tier nested navigation architecture (Stack, Drawer and Tab) to ensure a fluid user experience. By integrating intuitive data entry forms, real-time cloud synchronization via WebSockets and REST APIs , and local-first SQLite data persistence, this app also provides a highly responsive and secure financial management experience. The primary objective of this app is to offer users a clear, visual representation of their financial health, enabling better budgeting and financial decision-making on the go.

KEY FEATURES

This application incorporates several advanced technical and user-centric features:

Core Financial Management:

- **Comprehensive CRUD Operations:**

Users can effortlessly create, read, update, and delete their transaction records directly with instantaneous UI feedback.

- **Multi-Wallet Management:**

This app provides multiple wallets to view user financial health globally or drill down into specific wallets. User also can create and manage multiple wallets with custom names and icons to keep their different financial streams separate.

- **Smart Budgeting**

This app will provide a visual progress bars that show how much of your monthly budget you've consumed. When it exceeds 85% of user allocated spending, the progress colour will change to red to alert the user the “Near Budget” overrun.

- **Comprehensive Transaction Management**

Each transaction will be record and display at the full searchable history of all transactions with filtering capabilities. Each transaction can also be categorized using a pre-defined set of vibrant, easy-to-read categories and emojis.

Advanced Insights & Analytics:

- **Financial Calendar:**

A dedicated analytics dashboard featuring a calendar that show spending of each day and a dynamic pie chart that automatically aggregate and categorize spending data for deeper financial insights.

- **Real-Time Cloud Synchronization:**

Powered by a robust backend, ensuring that users' financial data and wallets are securely backed up and synchronized continuously to prevent data loss.

- **Dynamic UI/UX:**

The app would also provide a fully integrated theme system that adapts the UI for comfortable viewing in both light and dark environments. It's powered by *react-native-reanimated* and *Animated* API for smooth transition and interactive feedback.

Security & Technical Highlights:

- **Enterprise-Grade Security:**

A dedicated “Secure Access” layer that protects your financial data from unauthorized local access by using the security PIN lock. Besides that, secure session management via JWT tokens to ensure your profile and data remain private.

- **Data Persistence & Offline Fallback:**

This app will be utilizing *AsyncStorage*, the application caches recent **transaction history**, allowing users to view their financial records even without an active internet connection

- **Optimized Performance:**

The frontend utilizes React Native's *InteractionManager* to ensure lag-free transitions and animations.

Part A

SCREENS, COMPONENTS AND REACT-NAVIGATION

A.1 Screens & Layout

The Personal Financial Tracker application is designed with a premium, dark-themed Neo-Banking aesthetic, prioritizing readability, focus and a seamless user experience. The structural layout of each screens is engineered to be highly responsive, adapting smoothly to various mobile device dimensions and screen ratios.

1. Flexbox-Driven Responsive Layout

The layout structure is relies on React Native's *Flexbox* model. By utilizing properties such as *flex: 1*, *justifyContent*, and *alignItems*, the application ensures that UI elements such as the total balance summary cardds, transaction list rows, and the input forms are dynamically proportioned. This prevents the UI conflicts and ensures the consistent visual hierarchy across both small and large devices.

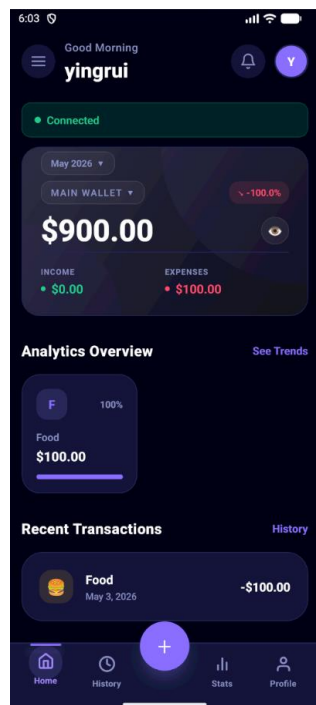
2. Component Grouping and Card Interface

To manage complex financial data visually, the layout employs a “Card-based” interface. Related information is grouped within distinct containers featuring rounded corners and subtle backgrounds. This creates clear visual boundaries, making the data highly scannable for user.

3. Core Screen Architecture

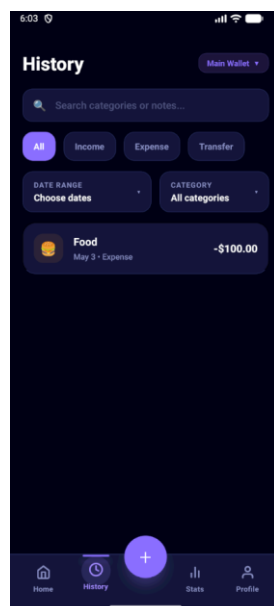
- **Dashboard Screen:**

Utilizes a vertical layout focusing on a hero section for the total balance, followed by a summarizes list of recent transactions.



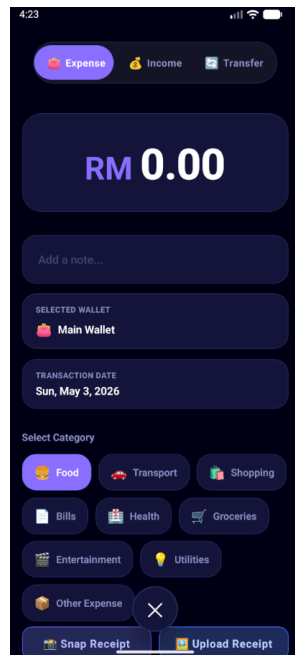
- **History Screen:**

Employs a full-screen *FlatList* Layout for scrolling through extensive lists of data.

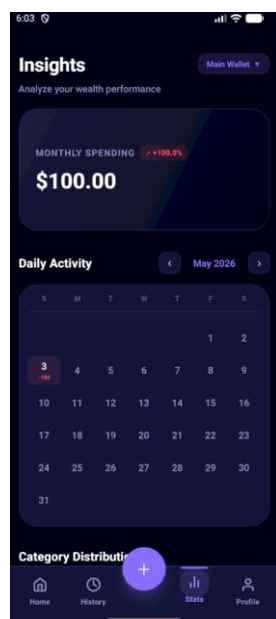


- **Add Transaction Screen:**

Structured as a clean, highly legible data entry form with large touch targets and floating input fields, ensure ease of use during one-handed operation.

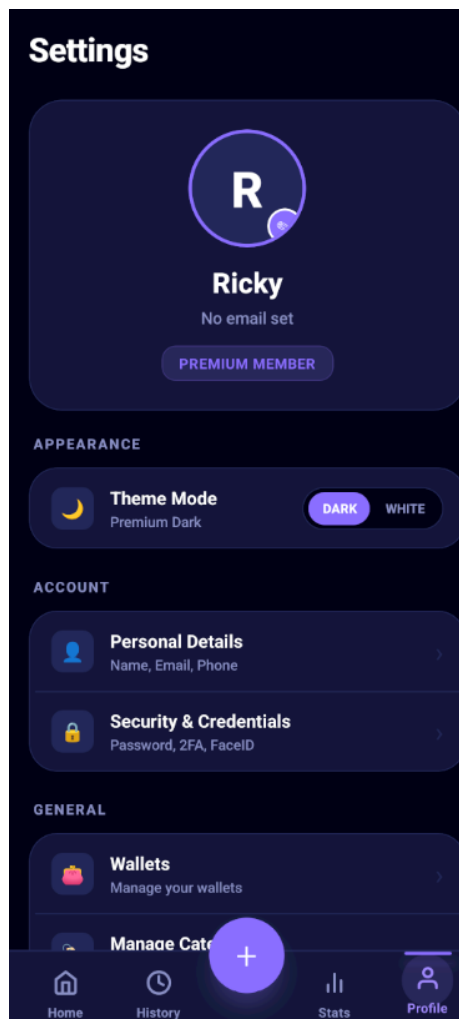


- Analytics Screen:**
 Features a specialized layout designed to allocate for data visualization components.



- Profile Screen:**

Profile screen feature enable user to set up their profile and security preferences and many more.



←

Personal Details

PROFILE PICTURE

R

Upload Photo

USERNAME

Ricky

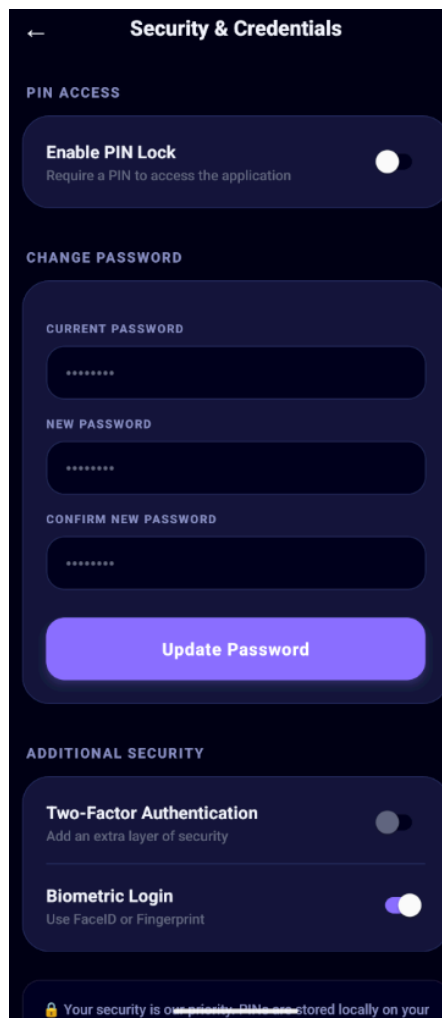
EMAIL ADDRESS

Enter email

PHONE NUMBER

Enter phone number

Save Changes



1. Use of Components

LoginScreen:

- **TextInput:** Used for capturing the user's email/username and password.
- **Pressable / TouchableOpacity:** Creates interactive elements for the "Login", "CreateAccount", and "Forgot Password" actions.
- **KeyboardAvoidingView:** Ensures the input fields are not obscured when the software keyboard opens.

- **ScrollView:** Enables vertical scrolling so the layout remains accessible on smaller devices.
- **Alert:** Displays error feedback (e.g., wrong credentials) and success messages.

RegisterScreen:

- **TextInput:** Used for the registration form (username, email, password, and confirm password).
- **Pressable / TouchableOpacity:** Used for the primary "Sign Up" button and the navigational "Login" link.
- **Alert:** Shows form validation warnings (e.g., passwords don't match) and successful registration alerts.
- **ScrollView:** Allows smooth scrolling through the comprehensive form layout. **Svg / Path:** (via react-native-svg) Renders the custom visibility "eye" icon for the password fields.

Dashboard Screen (Core View):

- **ScrollView:** Enables scrolling through the dashboard content (Total Balance, Analytics

Overview, Recent Transactions).

- **View / Text:** Core structural containers and typography components for presenting financial data.
- **Pressable:** Wraps clickable elements like "See Trends", wallet selector dropdowns, and

individual transaction rows.

- **ActivityIndicator:** Displays a loading spinner while fetching the initial financial data

from the backend.

- **Animated:** Powers the dynamic, smooth transition effects and hero balance mounting animations.

AddTransaction Screen:

- **TextInput:** Provides input fields for entering the transaction amount and optional custom notes.
- **Pressable:** Creates large, tappable zones for selecting Categories, transaction Types (Income/Expense), and the main Save button.
- **ScrollView:** Allows users to scroll through the full transaction form and the category selection grid.
- **Image:** Displays the thumbnail preview of the attached receipt.
- **PanResponder & Animated:** Handles swipe-to-dismiss gestures and smooth modal transitions.

History Screen:

- **FlatList:** (Note: Highlight this for extra marks!) Specifically implemented to render long lists of financial transactions efficiently with recycling, rather than a standard ScrollView.
- **View / Text:** Structures the transaction rows, displaying the date, category, and formatted amount.
- **Pressable:** Allows the user to tap on specific list items to navigate to the Transaction Detail view.

2. Reusability (Custom Components)

Reusability is used to avoid the rewriting of the same UI and logic multiple times, especially in insert or update workflows. In the Personal Finance Tracker, the reusable custom component is shared to all the main screens, so that users can get the same navigation and quick add behaviour when they are performing the transaction action. The main reusable component is `FinanceTabBar.tsx` in the `components/FinanceTabBar.tsx` and is plugged into the `MainTabsNavigator` once to serve all the main tabs.

In `MainTabsNavigator.tsx`, it reuses one of the custom tab bars for all pages:

```

<FinanceTabBar
  state={state as any}
  descriptors={{}} as any}
  navigation={mockTabBarNavigation as any}
/>

```

The custom component is used:

FinanceTabBar.tsx used in MainTabsNavigator.tsx are shared across the four main pages: Dashboard, History, Analytics, Profile.

In this mean that the tab UI is implemented once and the reused everywhere instead of creating separate tab controls inside each of the screen.

The reusability supports inserts/updates:

Insert Flow:

The floating action button (+) inside the FinanceTabBar.tsx navigates to the AddTransaction.tsx giving users a consistent way to insert the new records from the main tab. This ensure that the “Insert” workflow is globally accessible without of the redundant button in the Dashboard, History or Analytics UI code.

In FinanceTabBar.tsx reusable insert entry point (+ button) to the AddTransaction.tsx:

```

<Pressable
  style={[styles.fab, { backgroundColor: colors.card }]}
  onPress={() => {
    const parentNav = navigation.getParent();
    if (parentNav) {
      const { width, height } = Dimensions.get('window');
      const fabCenterY = height - (Platform.OS === 'ios' ? 76 : 72);
      (parentNav as any).navigate('AddTransaction', {
        fromFab: true,
        originX: width / 2,
        originY: fabCenterY,
        fast: true, // Hint for snappier animation
      });
    }
  }}
>
  <View style={[styles.fabInner, { backgroundColor: colors.primary, shadowColor: colors.primary }]}>
    <Text style={styles.fabIcon}>+</Text>
  </View>
</Pressable>

```

Update Flow:

The same reusable tab bar that provides the consistent navigation to data related screens such as History and Profile where users that access and perform the update or edit behaviours.

Reusable of interaction behaviour:

AnimatedTabButton is reused for every tab item, so that all tabs can share the same press animation. This creation of the isolated functional component to abstracts the styling, press feedback and animated micro interactions such as scaling and translating on press. It used to every tab item, so that it never duplicates the animation logic.

In FinanceTabBar.tsx of the reusable tab button abstraction:

```
function AnimatedTabButton({
  isFocused,
  label,
  onPress,
  routeName,
  onLayout,
  activeColor,
  inactiveColor,
  indicatorBg,
}): {
  isFocused: boolean;
  label: string;
  onPress: () => void;
  routeName: string;
  onLayout?: (event: LayoutChangeEvent) => void;
  activeColor: string;
  inactiveColor: string;
  indicatorBg: string;
} {
  const scale = useRef(new Animated.Value(1)).current;
  const translateY = useRef(new Animated.Value(0)).current;

  useEffect(() => {
    Animated.spring(translateY, {
      toValue: isFocused ? -4 : 0,
      useNativeDriver: true,
      friction: 8,
      tension: 100,
    }).start();
  }, [isFocused]);

  const handlePressIn = () => {
    Animated.spring(scale, {
      toValue: 0.9,
      useNativeDriver: true,
    }).start();
  };
}
```

```
const handlePressOut = () => {
  Animated.spring(scale, {
    toValue: 1,
    useNativeDriver: true,
  }).start();
};

return (
  <Pressable
    onPress={onPress}
    onPressIn={handlePressIn}
    onPressOut={handlePressOut}
    onLayout={onLayout}
    style={styles.tabButtonShell}
  >
    <Animated.View style={[styles.tabButton, { transform: [{ scale }, { translateY }] }]}>
      <View style={[styles.activeIndicatorBg, isFocused && styles.activeIndicatorBgVisible, { backgroundColor: indicatorBg }]} />
      <TabIcon routeName={routeName} color={isFocused ? activeColor : inactiveColor} />
      <Text numberOfLines={1} style={[styles.label, { color: isFocused ? activeColor : inactiveColor }]}>
        {label}
      </Text>
    </Animated.View>
  </Pressable>
);
```

Reusable tab metadata:

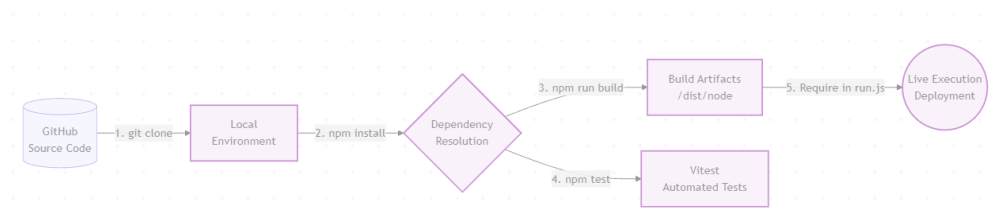
tabMeta is centralizes the tab labels in one location to reduce the hardcoded duplicates and make the future UI changes easier.

In FinanceTabBar.tsx the reusable tab label mapping in one place:

```
const tabMeta: Record<string, { label: string }> = {
  Dashboard: { label: 'Home' },
  History: { label: 'History' },
  Analytics: { label: 'Stats' },
  Profile: { label: 'Profile' },
};
```

3. Implementation of React-Navigation

To provide a seamless and scalable user experience, the application implements a sophisticated 3-tier nested navigation architecture utilizing the **@react-navigation** library. Rather than relying entirely on default navigator templates, we engineered a custom solution that combines a standard Stack Navigator with a highly customized Tab Navigator implementation. This structure ensures logical screen hierarchy, fluid swipe transitions, and centralized control over the application's routing state.



1. Stack Navigator

The foundation of our navigation is the **RootStackNavigator**, implemented using **@react-navigation/native-stack**. This acts as the primary router, handling top-level deep-linking, authentication flow and pushing full-screen modals.

For instance, pushing the **AddTransactionScreen** on top of the current view. This implements a Last-in-First-out (LIFO) stack, allowing users to intuitively navigate back to previous screen using the back button.

```
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import MainTabsNavigator from './MainTabsNavigator';
import AddTransaction from '../screens/AddTransaction';
import LoginScreen from '../screens/LoginScreen';

export type RootStackParamList = {
  Login: { prefillEmail?: string; registeredName?: string } | undefined;
  Register: undefined;
  ForgotPassword: { prefillUsername?: string } | undefined;
  HelpSupport: undefined;
  PrivacyPolicy: undefined;
  MainTabs: undefined;
  AddTransaction: { fromFab?: boolean; originX?: number; originY?: number } | undefined;
};

return (
  isLoading ? (
    <View style={{ flex: 1, backgroundColor: '#090a1f', justifyContent: 'center', alignItems: 'center' }}>
      <ActivityIndicator size="large" color="#7f5bff" />
    </View>
  ) : (
    <Stack.Navigator
      initialRouteName={initialRoute}
      screenOptions={{
        headerTintColor: '#f5f7ff',
        headerStyle: { backgroundColor: '#090a1f' },
        headerShadowVisible: false,
        contentStyle: { backgroundColor: '#090a1f' },
      }}
    >
      <Stack.Screen name="Login" component={LoginScreen} options={{ headerShown: false }} />

      <Stack.Screen
        name="MainTabs"
        component={MainTabsNavigator}
        options={{ headerShown: false }}
      />
      <Stack.Screen
        name="AddTransaction"
        component={AddTransaction}
        options={{
          headerShown: false,
          animation: 'none',
          presentation: 'transparentModal',
          contentStyle: { backgroundColor: 'transparent' }
        }}
      />
    </Stack.Navigator>
  )
);
```

2. Drawer Navigator

To provide a seamless and scalable user experience, this application implements a *DrawerNavigator* utilizing *@react-navigation/drawer*. This drawer serves as a persistent, globally accessible menu system for secondary application settings and user profile management without cluttering the primary bottom tab interface

This Drawer acts as the crucial middle layer. It is invoked by the *RootStackNavigator* and structurally nets the *MainTabsNavigator* as its primary default route. This ensures the user can intuitively swipe from any main dashboard screen to access the drawer function.

DrawerNavigator.tsx

```
export type DrawerParamList = {
  MainTabs: undefined;
  ProfileDetails: undefined;
  SecuritySettings: undefined;
  HelpSupport: undefined;
};

export default function DrawerNavigator() {
  const { colors } = useTheme();

  return (
    <Drawer.Navigator
      screenOptions={{
        headerShown: false,
        drawerStyle: { backgroundColor: colors.card },
        drawerActiveTintColor: colors.primary,
        drawerInactiveTintColor: colors.textMuted,
        drawerLabelStyle: { fontWeight: '600' },
      }}
    >
      <Drawer.Screen
        name="MainTabs"
        component={MainTabsNavigator}
        options={{ title: 'Home' }}
      />
    </Drawer.Navigator>
  );
}
```

3. Tab Navigator & PageView

Instead of using the standard *@react-navigation/bottom-tabs*, we implement a highly customized *MainTabNavigator*. To achieve smooth, native-feeling swipe gestures between main screens, we utilized *react-native-pager-view*.

This *PagerView* is then synchronized with our custom reusable component, *FinanceTabBar* hooks, we created a *PageWrapper* bridge. This custom implementation prevents UI locking during heavy data renders and allows tabs to share a single animated interaction logic

```
import React, { useRef, useState, useMemo, useEffect, createContext, useContext } from 'react';
import { View, StyleSheet } from 'react-native';
import PagerView from 'react-native-pager-view';
import Dashboard from '../screens/Dashboard';
import History from '../screens/History';
import Analytics from '../screens/Analytics';
import Profile from '../screens/Profile';
import FinanceTabBar from '../components/FinanceTabBar';
```

```
export default function MainTabsNavigator({ navigation }: any) {
  const [index, setIndex] = useState(0);
  const pagerRef = useRef<PagerView>(null);

  const routes = useMemo(() => [
    { name: 'Dashboard', key: 'Dashboard' },
    { name: 'History', key: 'History' },
    { name: 'Analytics', key: 'Analytics' },
    { name: 'Profile', key: 'Profile' },
  ], []);
```

```
  return (
    <PagerContext.Provider value={{ jumpTo }}>
      <View style={styles.container}>
        <PagerView
          ref={pagerRef}
          style={styles.pager}
          initialPage={0}
          onPageSelected={(e) => {
            const newIndex = e.nativeEvent.position;
            setIndex(newIndex);
          }}
        />
      </View>
    </PagerContext.Provider>
  );
```

```
    <View key="Dashboard" style={styles.page}>
      <PageWrapper
        component={Dashboard}
        isFocused={index === 0}
        navigation={navigation}
        route={routes[0]}
      />
    </View>
```

```
    <FinanceTabBar
      state={state as any}
      descriptors={{}} as any
      navigation={mockTabBarNavigation as any}
    />
  </View>
</PagerContext.Provider>
```

Part B

DATA PERSISTENCE

B.1 Usage of Data Persistence

The Personal Finance Tracker app relies on data persistence as a foundational mechanism to ensure that all financial information remains accessible across app sessions, device restarts, and server reboots.

The following types of data are persisted:

- Transactions: records of income and expenses linked to a wallet and category.
- Users: account information including credentials for authentication.
- Wallets: the financial accounts that belong to each user.
- Budgets: spending limits configured per category and time period.
- Recurring transactions: rules that automatically generate transactions on a set interval.
- Authentication sessions: tokens that keep a user securely logged in.

The app also handles sensitive user account information and authentication sessions. To protect this sensitive data and align with industry-standard practices, user authentication state is persisted through dedicated sessions and users tables, and passwords are encrypted using the scrypt hashing algorithm prior to storage. This ensures that plaintext credentials are never written to disk.

To manage this data reliably, the app employs a two-tier persistence architecture: a server-side SQLite database for authoritative data management and a client-side SQLite store for offline functionality.

1. Server-Side Persistent Storage

On the server side, the primary persistent store is a file-based SQLite database located at `finance_tracker.sqlite`. Managed by the backend service (`service.js`), this database acts as the single source of truth for all authoritative reads and

writes. Because it is file-based, data remains intact even during server reboots. Furthermore, this backend architecture is designed with robust operational and maintenance considerations. Schema migrations and structural alterations—such as adding new columns—are performed automatically at server startup through `ensureTables` and `column-check` routines, ensuring backward compatibility as new features are introduced. For debugging and auditing during development, the database file can also be externally backed up and inspected using provided tooling (`inspect_db.js`).

2. Client-Side Persistent Storage

On the mobile client, the application utilizes `react-native-sqlite-storage` (implemented via `sqlite.ts`) for local on-device persistence of transaction records. This client-side relational database is essential for enabling full offline CRUD operations, allowing users to seamlessly record and browse transactions without an active network connection. Once network connectivity is restored, the mobile app synchronises with the server-side endpoints, ensuring the backend database remains the definitive source of truth across multiple devices. In development or testing environments where the native SQLite module is unavailable, the client gracefully falls back to an in-memory store to keep the app functional. Finally, while `@react-native-async-storage/async-storage` is available within the project, its role is strictly limited to lightweight key-value caching and basic application settings, reserving the robust SQLite implementation entirely for primary transactional data.

B.2 Implementation of Data Persistence

The implementation of data persistence in the Personal Finance Tracker employs a structured, multi-layered approach that ensures both data durability and application reliability. This section details how SQLite persistence is integrated, configured, and maintained across the server and mobile client.

1. Server-Side Implementation

The backend service, managed by `service.js`, initializes and maintains the SQLite database through a series of setup routines executed at server startup. The `ensureTables()` function programmatically creates the required database schema if it does not already exist, including six core tables: `'users'`, `'sessions'`, `'password_resets'`, `'wallets'`, `'budgets'`, `'transactions'`, and `'recurring_transactions'`. Each table is defined with strict schema constraints: primary keys for uniqueness, foreign-key relationships for referential integrity, and CHECK constraints to enforce valid data types (for example, the `'type'` column in `transactions` is restricted to `'income'`, `'expense'`, or `'transfer'`). Additionally, indexes are created on frequently queried columns, such as `'idx_transactions_user_date'`, to optimize retrieval performance for large datasets.

```
function ensureTables(onDone) {
  const db = openDb();
  db.serialize(() => {
    db.run('PRAGMA foreign_keys = ON');

    // Auth tables
    db.run(`
      CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT NOT NULL UNIQUE,
        password TEXT NOT NULL,
        email TEXT,
        phone TEXT,
        profilePic TEXT,
        createdAt TEXT NOT NULL
      )
    `);
  });
}
```

```
db.run(`
  CREATE INDEX IF NOT EXISTS idx_transactions_user_date
  ON transactions(userId, date DESC)
`);
```

To maintain backward compatibility as the application evolves, the server performs automated schema migrations at startup through column-check routines. For instance, the `ensureTransactionColumns()` function inspects the existing schema using PRAGMA directives and dynamically adds new columns

(such as ‘note’ or ‘receiptUrl’) if they are missing, allowing existing databases to upgrade gracefully without manual intervention or data loss.

```
function ensureTransactionColumns(db, onDone) {
  db.all('PRAGMA table_info(transactions)', [], (err, rows) => {
    if (err) {
      console.error(err.message);
      if (typeof onDone === 'function') {
        onDone();
      }
      return;
    }

    const existingColumns = new Set((rows || []).map(column => column.name));
    const alterStatements = [];
```

Data integrity is enforced through parameterized SQL queries throughout the backend. All user inputs—whether from authentication requests or transaction creation—are bound as query parameters rather than concatenated into SQL strings. This practice prevents SQL injection vulnerabilities and ensures that data is properly escaped and typed. For example, transaction creation uses prepared statements with parameter placeholders (“?”), and all inputs undergo validation before database insertion.

```
// Transaction Endpoints
app.get('/api/transactions', requireAuth, (req, res) => {
  const { walletId } = req.query;
  const db = openDb();
  let sql = 'SELECT * FROM transactions WHERE userId = ?';
  let params = [req.auth.userId];

  if (walletId) {
    sql += ' AND walletId = ?';
    params.push(walletId);
  }

  sql += ' ORDER BY date DESC';
```

Password security is implemented using the scrypt key-derivation function. When a user registers or changes their password, the plaintext password is hashed with a randomly generated salt, and only the salt-hash pair is stored in the database. Authentication is verified by re-hashing the provided password with the stored salt and comparing the result using a timing-safe comparison function to prevent timing attacks.

```
function hashPassword(password) {
  const salt = crypto.randomBytes(16).toString('hex');
  const hash = crypto.scryptSync(password, salt, 64).toString('hex');
  return `${salt}:${hash}`;
}

function verifyPassword(password, stored) {
  const parts = String(stored || '').split(':');
  if (parts.length !== 2) {
    return stored === password;
  }
}
```

Authentication state is managed through the ‘sessions’ table, which stores login tokens linked to user IDs and creation timestamps. Tokens are cryptographically generated using ‘crypto.randomBytes()’ to ensure uniqueness and unpredictability. Upon login, a new session record is created; upon logout, the corresponding record is deleted. The backend validates every authenticated request by querying the sessions table to verify the provided bearer token.

```
function generateToken() {
  return crypto.randomBytes(24).toString('hex');
}
```

```
function requireAuth(req, res, next) {
  const token = getBearerToken(req);
  if (!token) {
    return res.status(401).json({ message: 'Missing auth token' });
  }

  const db = openDb();
  db.get(
    `SELECT sessions.userId, users.username
    FROM sessions
    JOIN users ON users.id = sessions.userId
    WHERE sessions.token = ?`,
    [token],
    (err, row) => {
      closeDb(db);

      if (err) {
        return res.status(500).json({ message: 'Database error' });
      }

      if (!row) {
        return res.status(401).json({ message: 'Invalid or expired token' });
      }

      req.auth = {
        token,
        userId: Number(row.userId), // Store as Number for consistent DB querying
        username: row.username,
      };
      return next();
    }
  );
}
```

2. Client-Side Implementation

On the mobile client, persistent storage is implemented through `sqlite.ts`, which wraps the ‘`react-native-sqlite-storage`’ module. The client initializes the database by calling ‘`openDatabase()`’, which establishes a connection to a named database stored on the device's local file system. The schema is similar to the server schema but focused on the transactions table, allowing users to store transaction records locally.

```
const isJestRuntime = typeof process !== 'undefined' && Boolean(process.env.JEST_WORKER_ID);
const hasSQLiteModule = Boolean(SQLiteModule?.openDatabase) && !isJestRuntime;
let sqliteNativeUnavailable = false;

async function openDatabase(): Promise<any> {
  if (!hasSQLiteModule || sqliteNativeUnavailable) {
    return null;
  }

  if (!dbConnectionPromise) {
    try {
      dbConnectionPromise = SQLiteModule.openDatabase({
        name: config.sqliteDbName,
        location: 'default',
      });
    } catch {
      sqliteNativeUnavailable = true;
      dbConnectionPromise = null;
      return null;
    }
  }

  return dbConnectionPromise;
}
```

The client module implements a graceful fallback mechanism: if the native SQLite module is unavailable (for example, during Jest testing or if native module linking fails), the application falls back to an in-memory array-based store. This fallback is transparent to the calling code and allows the application to remain functional across different environments without code modification.

```
try {
  // Use runtime require to avoid breaking Jest (native module unavailable there).
  // eslint-disable-next-line @typescript-eslint/no-var-requires
  SQLiteModule = require('react-native-sqlite-storage');
  if (typeof SQLiteModule?.enablePromise === 'function') {
    SQLiteModule.enablePromise(true);
  }
} catch {
  SQLiteModule = null;
}
```

Data initialization on the client is handled by ‘`initDatabase()`’, which ensures the transactions table exists and populates it with seed data if configured. All

client-side CRUD operations are performed asynchronously using promises, ensuring that long-running database operations do not block the user interface.

```
export async function insertTransaction(input: CreateTransactionInput): Promise<Transaction> {
  const normalized = toDbTransaction(input);
  const newRow: Transaction = {
    ...normalized,
    id: nextId(),
  };

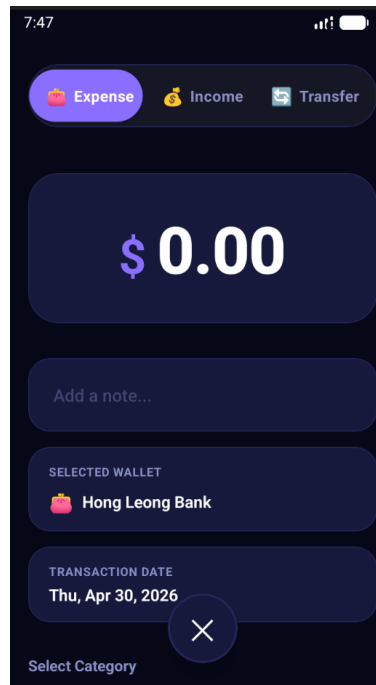
  if (!hasSQLiteModule || sqliteNativeUnavailable) {
    memoryStore = sortTransactions([newRow, ...memoryStore]);
    await emitChange();
    return newRow;
  }

  await initDatabase();
  await executeSql(
    `INSERT INTO ${TABLE_NAME} (id, amount, type, category, date, note, receiptUrl, userId)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?)`,
    [
      newRow.id,
      newRow.amount,
      newRow.type,
      newRow.category,
      newRow.date,
      newRow.note ?? '',
      newRow.receiptUrl ?? '',
      newRow.userId,
    ],
  );

  await emitChange();
  return newRow;
}
```

Input validation and data normalization occur before insertion. For instance, the ‘toDbTransaction()’ function normalizes transaction amounts to numbers, ensures optional fields (like notes and receipt URLs) have default values, and applies consistent formatting to prepare data for reliable storage.

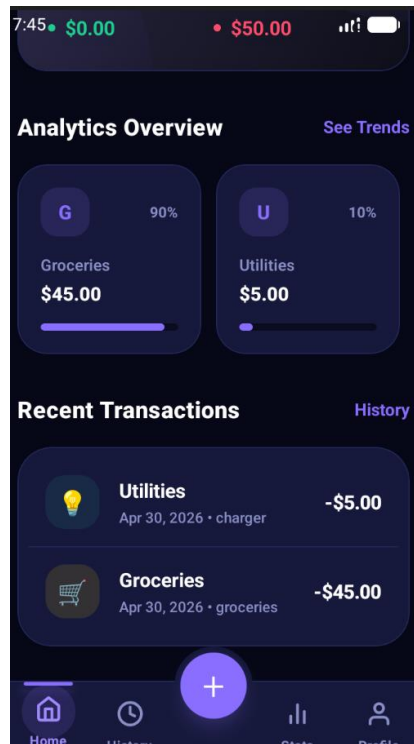
```
function toDbTransaction(input: CreateTransactionInput): CreateTransactionInput {
  return {
    ...input,
    amount: Number(input.amount),
    note: input.note ?? '',
    receiptUrl: input.receiptUrl ?? '',
  };
}
```



Offline-first architecture is supported through the client SQLite store: users can create, read, update, and delete transactions without a network connection. The client maintains a list of listeners (observer pattern) that are notified whenever data changes, allowing the UI to reflect updates in real time. When network connectivity is restored, a synchronization mechanism sends pending changes to the server, which acts as the authoritative source.

```
const seedRows: Transaction[] = [
  createSeedTransaction('seed-1', 4200, 'income', 'salary', '2026-04-04T08:30:00Z', 'Monthly salary'),
  createSeedTransaction('seed-2', 84.2, 'expense', 'groceries', '2026-04-04T11:45:00Z', 'Weekend grocery run'),
  createSeedTransaction('seed-3', 14.9, 'expense', 'transport', '2026-04-03T15:15:00Z', 'Grab ride'),
  createSeedTransaction('seed-4', 120.0, 'expense', 'utilities', '2026-04-02T09:00:00Z', 'Water bill'),
  createSeedTransaction('seed-5', 250.0, 'income', 'freelance', '2026-04-01T19:00:00Z', 'Side project payment'),
];

let memoryStore: Transaction[] = [...seedRows];
```



3. Error Handling and Robustness

Both the server and client implement error handling to manage database failures gracefully. On the server, if a database operation fails, the error is logged and an appropriate HTTP status code is returned to the client (for example, 500 for internal errors, 400 for validation failures). On the client, if SQLite operations fail, the application falls back to the in-memory store and continues functioning, ensuring that temporary database issues do not crash the application.

The implementation also includes audit and debugging features. The `inspect_db.js` utility provides developers with a command-line interface to query the server database directly, inspect table schemas, and verify data integrity during development and troubleshooting.

```
const sqlite3 = require('sqlite3').verbose();
const db = new sqlite3.Database('./db/finance_tracker.sqlite');

db.all("SELECT name, sql FROM sqlite_master WHERE type='table'", (err,
  if (err) {
    console.error(err);
    process.exit(1);
  }

  tables.forEach(table => {
    console.log(`\nTable: ${table.name}`);
    console.log('SQL:', table.sql);

    db.all('PRAGMA table_info(`${table.name}`)', (err, columns) => {
      console.log('Columns:', columns);
    });
  });
});
```

Through this comprehensive implementation, the Personal Finance Tracker achieves reliable, secure, and user-friendly data persistence that supports both online and offline workflows while maintaining data integrity and security.

B.3 CRUD Operations & Input Validation

B.3.1 Create

The create operation is implemented with using the offline-first approach to ensure the seamless user experience even without the active internet connection. In the backend section, the POST routes designed for creation such as the transaction endpoint in service.js that are protected by the shared requireAuth middleware. The server validates the incoming payload that to ensure the required fields such as the amount, type and category are present and correct, then execute the parameterized SQL queries to secure the insert the new record into the database that linked with the authenticated user's ID.

In service.js:

```
app.post('/api/transactions', requireAuth, (req, res) => {
  const { amount, type, category, date, note, receiptUrl, walletId } = req.body || {};
  if (!amount || !type || !category || !date) return res.status(400).json({ message: 'Missing fields' });

  const db = openDb();
  db.run(
    `INSERT INTO transactions(userId, amount, type, category, note, date, createdAt, receiptUrl, walletId)
    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)`,
    [req.auth.userId, Number(amount), type, category, note || '', date, new Date().toISOString(), receiptUrl || '', walletId || null],
    function(err) {
      closeDb(db);
      if (err) return res.status(500).json({ message: 'Database error' });
      emitBudgetAlertsForUser(req.auth.userId, date);
      res.status(201).json({ id: this.lastID, affected: this.changes });
    }
  );
});
```

In the client side, the creation flow is managed by local database wrappers and API services. When a user that submits a new transaction, the app immediate writes the record into the database. At the same time, it queues the action and

the triggers a background of the synchronization process of `syncPendingTransactions()`. This guarantees that the UI is updates instantly with the new data, the actual of REST API request to the server is reliably assign in the background as soon as the network connectivity is available.

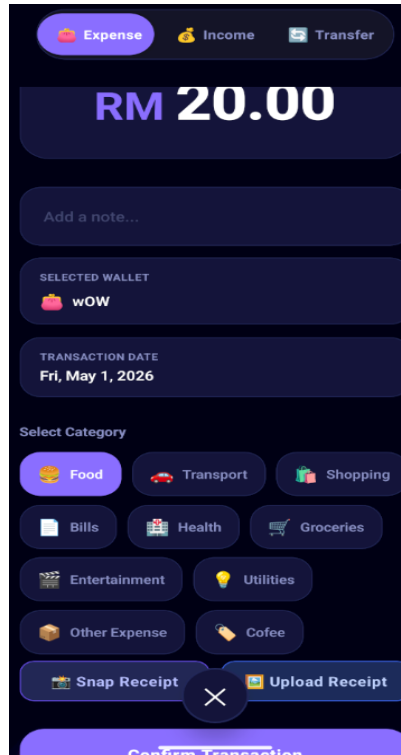
In `transactionApi.ts`:

```
export async function insertTransaction(input: CreateTransactionInput): Promise<Transaction> {
  const session = getAuthSession();
  const localTransaction = await insertTransactionLocal({
    ...input,
    userId: session?.userId ?? input.userId,
  });

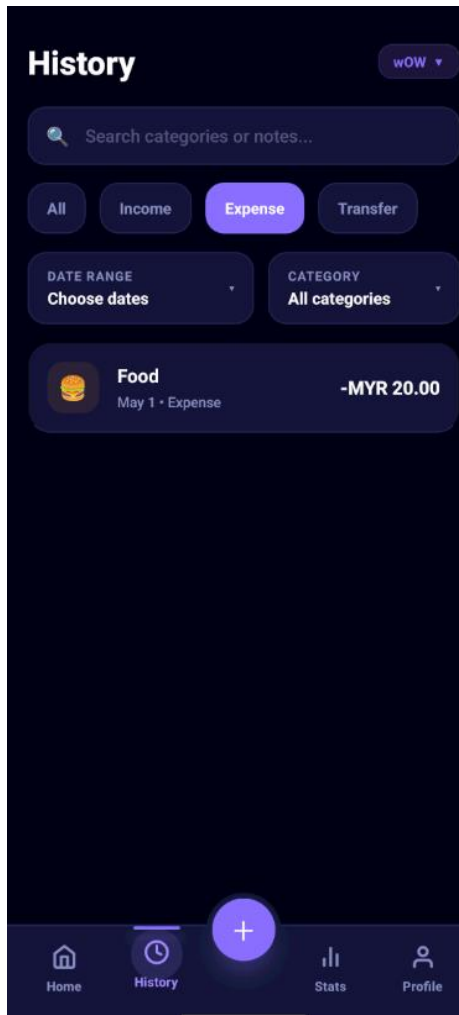
  await addPendingTransactionChange('create', localTransaction.id, localTransaction);
  setPendingCount((await getPendingTransactionChanges()).length);
  clearTransactionCache();
  void syncPendingTransactions();
  return localTransaction;
}
```

`syncPendingTransactions()`:

```
export async function syncPendingTransactions(): Promise<void> {
  if (isSyncInProgress) {
    return;
  }
}
```



After Confirm Transaction:



B.3.2 Read

The application uses a read-first CRUD flow for presenting financial data. On the backend, every read route is protected by authentication through the shared `requireAuth` middleware, so only the signed-in user can retrieve their own records. The main transaction read endpoint returns the user's transactions and supports an optional `walletId` filter. Similar read-side validation is applied to budget and analytics requests. Both require a `month` query parameter and reject the request if it is missing. The auth gate used by these routes is defined in `service.js`.

```
// Transaction Endpoints
app.get('/api/transactions', requireAuth, (req, res) => {
  const { walletId } = req.query;
  const db = openDb();
  let sql = 'SELECT * FROM transactions WHERE userId = ?';
  let params = [req.auth.userId];

  if (walletId) {
    sql += ' AND walletId = ?';
    params.push(walletId);
  }

  sql += ' ORDER BY date DESC';

  db.all(sql, params, (err, rows) => {
    closeDb(db);
    if (err) return res.status(500).json({ message: 'Database error' });
    res.status(200).json(rows || []);
  });
});

// Budget Endpoints
app.get('/api/budgets', requireAuth, (req, res) => {
  const month = (req.query.month || '').toString();
  if (!month) return res.status(400).json({ message: 'month query is required, format YYYY-MM' });

  const db = openDb();
  db.all(
    'SELECT id, category, month, amount, createdAt FROM budgets WHERE userId = ? AND month = ? ORDER BY category ASC',
    [req.auth.userId, month],
    (err, rows) => {
      closeDb(db);
      if (err) return res.status(500).json({ message: 'Database error' });
      res.status(200).json(rows || []);
    }
  );
});
```

On the client side, the read path is handled by `transactionApi.ts`. It first loads local SQLite data for offline access, then fetches the server version and merges both sources so the UI stays stable even when the network is slow or unavailable. If the server request fails, the app falls back to the local data instead of showing an error state. This makes the read operations reliable and ensures the dashboard and history screens can still display financial records when offline.

```
const fallbackUserId = getUserIdForFallback(_userId);
const localRows = fallbackUserId ? await getTransactionsByUserLocal(fallbackUserId) : await getAllTransactions();
const localData = walletId
  ? localRows.filter(item => String(item.walletId ?? '') === String(walletId))
  : localRows;

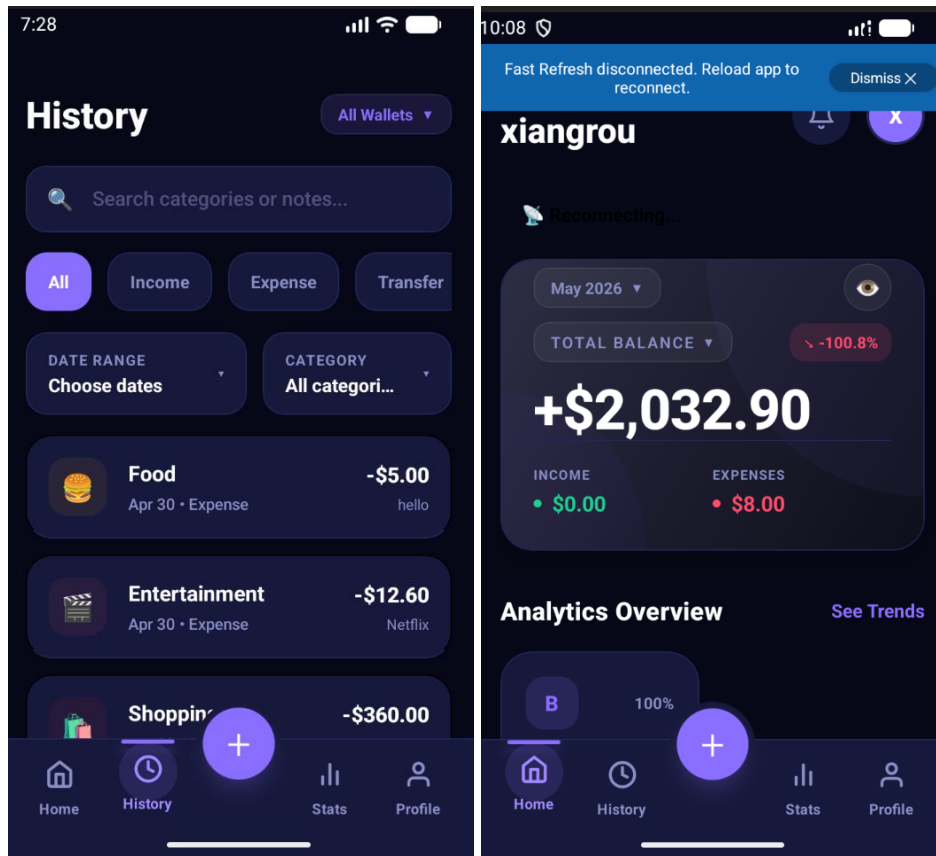
const pendingChanges = await getPendingTransactionChanges();

await syncPendingTransactions();

let url = `${config.apiUrl}/api/transactions`;
if (walletId) {
  url += `?walletId=${encodeURIComponent(walletId)}`;
}

try {
  const response = await fetch(url, {
    headers: {
      ...getAuthHeaders(),
    },
  });
}

const rows = await parseResponse<any[]>(response);
const serverData = rows.map(mapTransactionRow);
const filteredServerData = walletId
  ? serverData.filter(item => String(item.walletId ?? '') === String(walletId))
  : serverData;
```



B.3.3 Update

The update operation follows the same delayed, offline side architecture as the create and delete operation to guarantee the strong user experience across changes of network conditions.

On the backend, the PUT endpoint in `service.js` is secured by the `requireAuth` middleware, ensuring the user can only edit their own transaction records. The server utilizes a parameterized UPDATE SQL statement to sanitize the input such as the amount, category, and note, thereby protecting against SQL injection vulnerabilities. Once the database is successfully updated, the server recalculates the limit of metrics by invoking `emitBudgetAlertsForUser()` before returning to the success status.

In `service.js`:

```

app.put('/api/transactions/:id', requireAuth, (req, res) => {
  const { amount, type, category, date, note, receiptUrl, walletId } = req.body || {};
  const db = openDb();
  db.run(
    `UPDATE transactions SET amount = ?, type = ?, category = ?, date = ?, note = ?, receiptUrl = ?, walletId = ?
    WHERE id = ? AND userId = ?`,
    [Number(amount), type, category, date, note || '', receiptUrl || '', walletId || null, req.params.id, req.auth.userId],
    function(err) {
      closeDb(db);
      if (err) return res.status(500).json({ message: 'Database error' });
      emitBudgetAlertsForUser(req.auth.userId, date);
      res.status(200).json({ ok: true, affected: this.changes });
    }
  );
});

```

EmitBudgetAlertsForUser():

```

function emitBudgetAlertsForUser(userId, dateValue) {
  if (!financeNamespace) {
    return;
  }

  const targetDate = dateValue ? new Date(dateValue) : new Date();
  if (Number.isNaN(targetDate.getTime())) {
    return;
  }

  const monthKey = targetDate.toISOString().slice(0, 7);
  const db = openDb();

  db.all(
    `SELECT b.category, b.amount AS budgetAmount,
      COALESCE(SUM(CASE WHEN t.type = 'expense' THEN t.amount ELSE 0 END), 0) AS actualAmount
    FROM budgets b
    LEFT JOIN transactions t
      ON t.userId = b.userId
      AND t.category = b.category
      AND substr(t.date, 1, 7) = b.month
    WHERE b.userId = ? AND b.month = ?
    GROUP BY b.category, b.amount`,
    [userId, monthKey],
    (err, rows) => {
      closeDb(db);

      if (err || !rows || rows.length === 0) {
        return;
      }

      rows.forEach(row => {
        const budgetAmount = Number(row.budgetAmount) || 0;
        const actualAmount = Number(row.actualAmount) || 0;
        if (budgetAmount > 0 && actualAmount >= budgetAmount * 0.9) {
          financeNamespace.to(`user_${userId}`).emit('budget_alert', {
            category: row.category,
            spent: Math.round(actualAmount * 100) / 100,
            limit: Math.round(budgetAmount * 100) / 100,
            percentUsed: Math.round((actualAmount / budgetAmount) * 100),
            message: `${row.category} budget is ${Math.round((actualAmount / budgetAmount) * 100)}% used for ${monthKey}`,
          });
        }
      });
    }
  );
}

```

In client side, the update flow inside transactionApi.ts modifies the target row instantly in the local SQLite database(updateTransactionLocal()) so that the UI updates immediately. Instead of waiting the successful HTTP request, it logs an 'update' action along with the newly modified payload in the pending_transaction_changes table via addPendingTransactionChange(). The API cache is then cleared to seamlessly refresh the dashboard readings, the

syncPendingTransaction() is invoked in the background. When the internet connection is available, the sync layer loops over the pending queue and the reliably pushed the updated data to the server.

In transactionApi.ts:

```
export async function updateTransaction(
  id: string,
  updates: TransactionUpdate,
  fallback?: Transaction,
): Promise<Transaction | null> {
  const next = {
    amount: Number(updates.amount ?? fallback?.amount ?? 0),
    type: updates.type ?? fallback?.type,
    category: updates.category ?? fallback?.category,
    date: updates.date ?? fallback?.date,
    note: updates.note ?? fallback?.note ?? '',
    receiptUrl: updates.receiptUrl ?? fallback?.receiptUrl ?? '',
    walletId: updates.walletId ?? fallback?.walletId,
  };

  if (!next.type || !next.category || !next.date || next.amount <= 0) {
    throw new Error('amount, type, category and date are required');
  }

  const result = await updateTransactionLocal(id, updates);
  if (!result) {
    return null;
  }

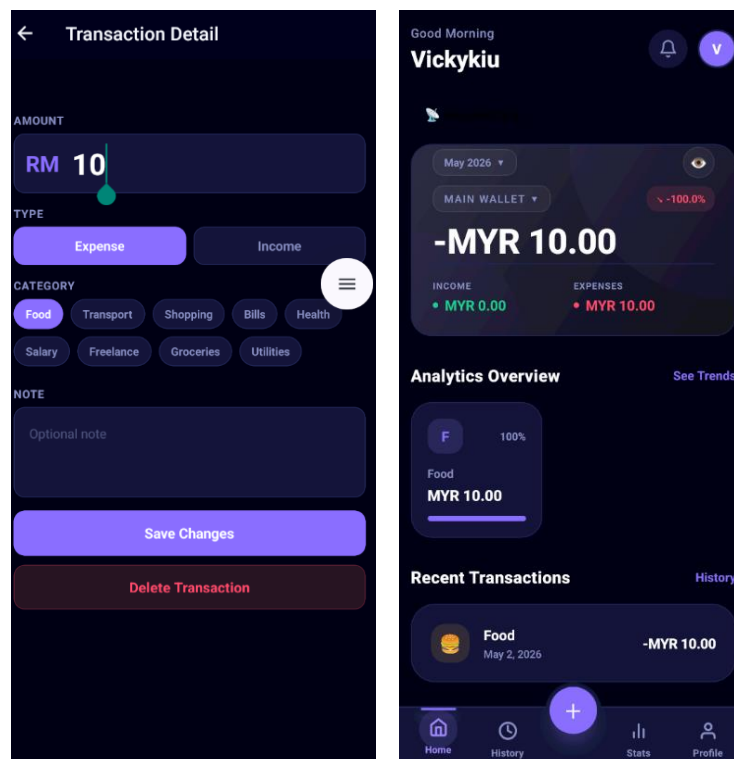
  await addPendingTransactionChange('update', id, {
    amount: result.amount,
    type: result.type,
    category: result.category,
    date: result.date,
    note: result.note ?? '',
    receiptUrl: result.receiptUrl ?? '',
    walletId: result.walletId,
  });

  clearTransactionCache();
  setPendingCount((await getPendingTransactionChanges()).length);
  void syncPendingTransactions();
  return result;
}
```

Before Update Transactions:



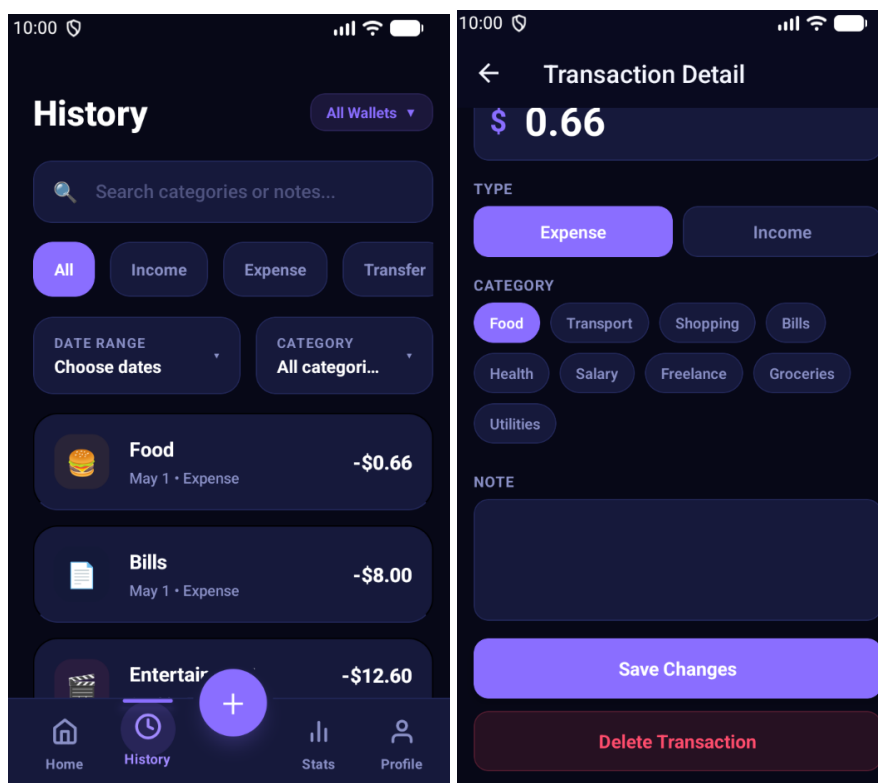
After Update:

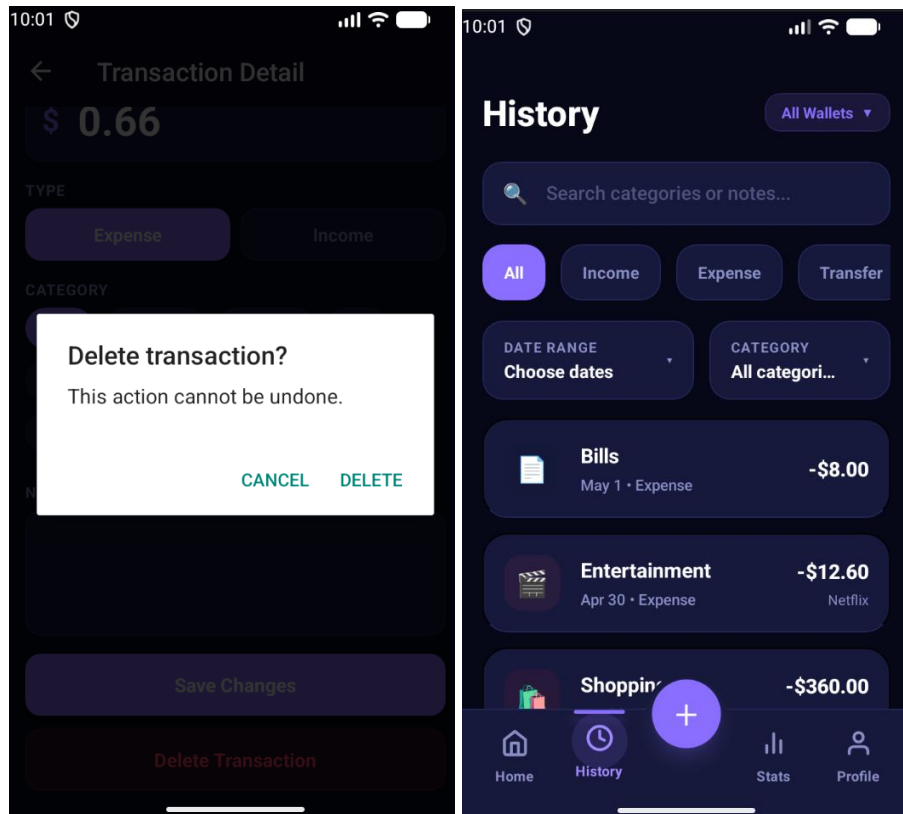


B.3.4 Delete

The delete operation is implemented as a deferred, offline-safe workflow. On the server, authenticated DELETE routes in service.js remove only the transaction that belongs to the current user.

On the client, `transactionApi.ts` does not delete the row from SQLite immediately. Instead, `deleteTransaction()` records a pending delete in `pending_transaction_changes`, clears the transaction cache, and triggers `syncPendingTransactions()`. When connectivity is available, the sync layer sends the DELETE request to the backend; after the server confirms success, the client removes the local SQLite row and clears the pending change. This approach preserves data consistency while still supporting offline use.





Part C

CLOUD CONNECTIVITY

C.1 Usage of Cloud Connectivity

The Personal Finance Tracker application heavily relies on cloud connectivity to ensure data synchronization across devices, deliver real-time notifications, and provide accurate, up-to-date financial contexts. The cloud connectivity features implemented in this application can be categorized into three primary usages:

- 1. REST API Data Synchronization:** The application uses a robust RESTful API architecture to synchronize local data (transactions, wallets, budgets, and user profiles) with the central backend server. This allows users to access their financial data seamlessly across multiple devices while maintaining a single source of truth in the cloud.
- 2. Real-Time WebSockets & Interactive Notifications:** To provide an interactive and responsive user experience, the application utilizes persistent WebSockets. This connection is crucial for delivering instant alerts, such as when a user's spending exceeds their allocated budget threshold, or when background recurring transactions are automatically synced. The application utilizes a highly decoupled Observer Pattern to broadcast these updates to the React UI in real time, persisting alerts into a dedicated notification centre.
- 3. External Currency System Integration:** To cater to international financial tracking, the app connects to third-party financial APIs (Alpha Vantage) to fetch real-time currency exchange rates. This allows the application to dynamically convert and display transaction amounts between USD and MYR based on live market data, rather than relying on static, outdated rates.

C.1.1 REST API Data Synchronization:

Purpose and Benefits

The REST API synchronization layer ensures that user financial data (transactions, wallets, budgets, and profiles) remains consistent across multiple devices while maintaining a reliable offline-first experience. Users can:

- Create, Read, Update, and Delete transactions seamlessly online, Auto synchronization for offline new transactions
- Access financial data from multiple devices with automatic synchronization
- Work offline without losing any data which the pending changes are queued locally
- Maintain data integrity through a smart merge strategy that respects server-as-source-of-truth while protecting local changes

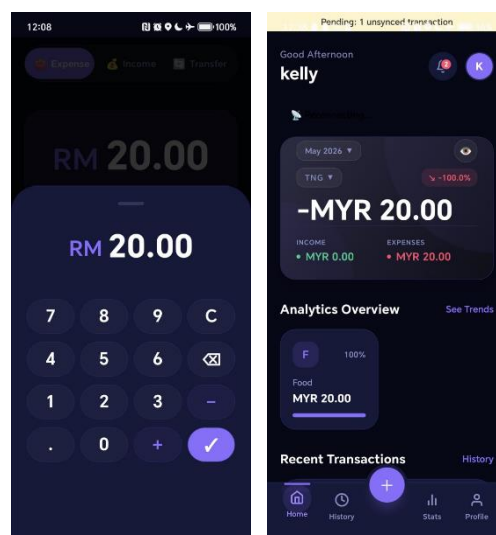
Key Features

Feature 1: Automatic Offline Support

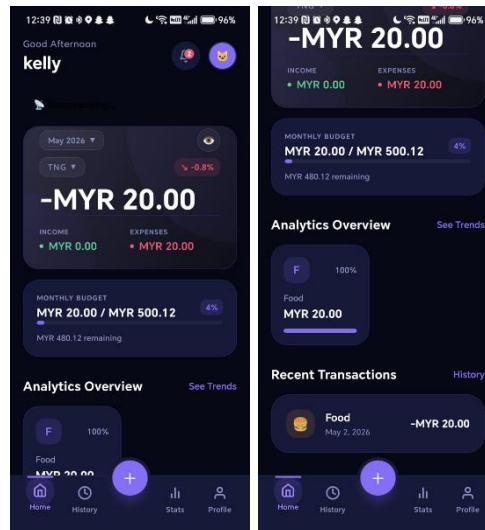
Users can add transactions even without internet connectivity. The application:

- Stores changes locally in SQLite database
- Queues pending operations in a dedicated change tracking table
- Automatically syncs when connectivity is restored

Add Transaction during Offline (Fly mode):



After Online (Auto-sync):



Feature 2: Multi-Device Synchronization

When a user logs in on a new device:

- Local transaction history is preserved
- Pending changes are synced first (maintains operation order)
- Server data is merged with local data
- Conflicts are resolved intelligently (local changes take priority for non-synced items)

Feature 3: Continuous Background Sync

The application runs a background sync process every 15 seconds:

- Monitors pending transaction changes
- Prioritizes older changes first (FIFO order)
- Updates pending count in UI for user awareness
- Stops if network errors occur to preserve operation order

C.1.2 Real-Time WebSocket Communication

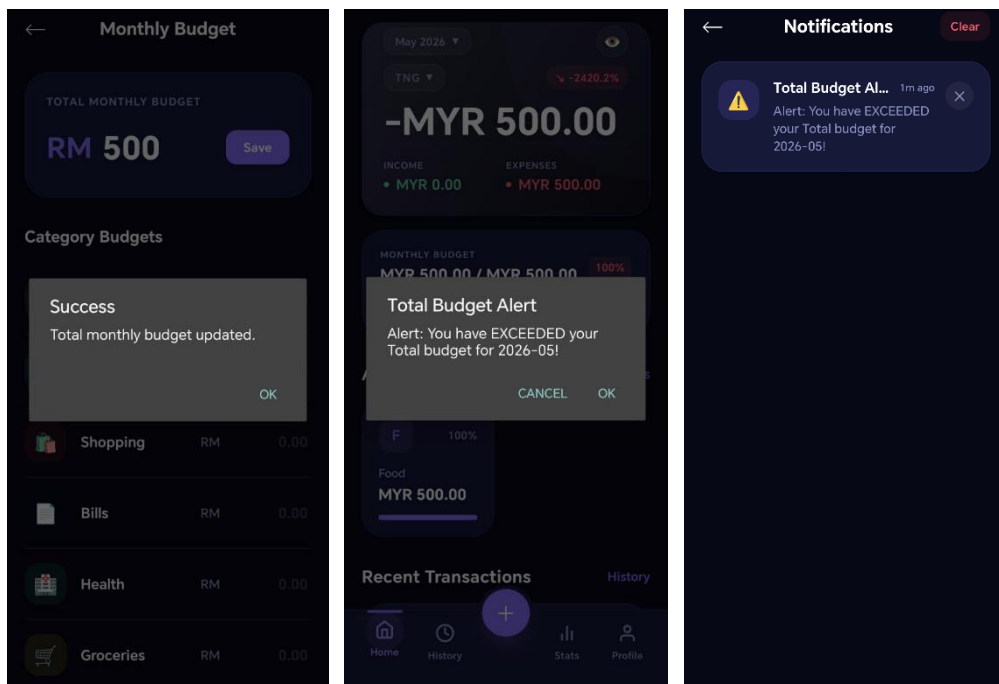
Purpose and Benefits

WebSockets provide instant, bidirectional communication between client and server, which have feature such as:

Feature 1: Budget Alert Notifications

When a user's spending in a category exceeds their budget which being set at setting:

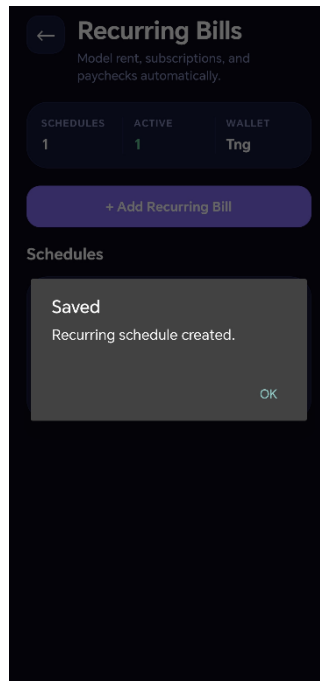
- Instant Alert: User receives immediate notification via WebSocket
- Actionable Information: Message helps user understand spending status immediately



Feature 2: Recurring Transaction Sync Notifications

When background recurring transactions are automatically synchronized:

- Sync Confirmation: Server notifies client of successful sync
- Count Information: User sees how many transactions were synced
- Persistent Storage: Notifications persist in notification centre



Feature 3: Connection Status Awareness

Application monitors WebSocket connection status:

- Displays online/offline indicator to user

```
C:\Users\Kelly\WAD_Assignment\Personal-Finance-Tracker\db>node service.js
🚀 Starting backend...
✓ Database tables created
✓ Database seeded
Server running on port 5001
Database: C:\Users\Kelly\WAD_Assignment\Personal-Finance-Tracker\db\finance_tracker.sqlite
✓ WebSocket listening on /finance namespace
✓ Backend ready for connections
[Socket] User connected: 74CiKEq4T9wJxtbcAAAB
[Socket] User 3 joined room
```

```
[Socket] User disconnected: mIIX7qTOWXMmWPdPAAAF
[Socket] User connected: a36mN2cEYb2uMGtaAAAH
[Socket] User 3 joined room
```

C.1.3 External Currency System Integration

Purpose and Benefits

The application integrates with Alpha Vantage API to provide real-time USD to MYR currency conversion:

Feature 1: Real-Time Exchange Rates

- Live Market Data: Exchange rates updated every 700s, 11.6 minutes from Alpha Vantage
- Automatic Conversion: Transactions automatically display in user's preferred currency
- Cached Rates: Falls back to cached rates during API outages
- Transparent Display: Users see both original amount and converted amount

Feature 2: Multi-Currency Support

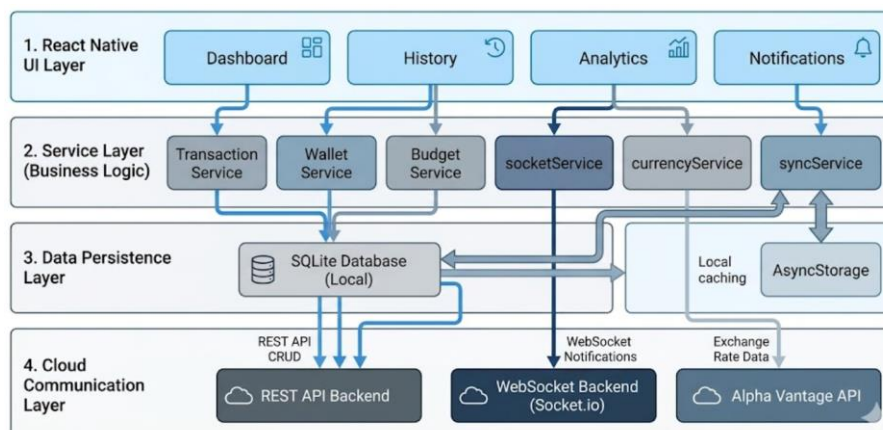
Users can:

- Set Preferred Currency: Choose between USD or MYR
- View Transactions: See all amounts in preferred currency
- Calculate Totals: Dashboard totals automatically use selected currency
- Maintain Accuracy: Conversion happens at calculation time using latest rates

Feature 3: Offline Fallback

- Last Known Rate: If API is unavailable, application uses last cached rate
- Graceful Degradation: Application continues functioning with cached data
- User Awareness: UI indicates when using cached (offline) rates
- Automatic Recovery: Retries fetching fresh rates when connectivity returns

C.2 Implementation of Cloud Connectivity



C.2.1 REST API Implementation

Configuration

The REST API base URL is configured in *appConfig.ts*:

- Supports both Android emulator and iOS simulator configurations
- Defaults to localhost for development
- Configurable via environment variables for different deployments
- Uses Flask backend running on port 5001

File: *config/appConfig.ts*

```
export const config = {
  appMode,
  isLocalMode: appMode === 'local',
  demoUserId: process.env.DEFAULT_USER_ID ?? 'demo-user',
  currencyCode: process.env.DEFAULT_CURRENCY_CODE ?? 'USD',
  sqliteDbName: process.env.SQLITE_DB_NAME ?? 'finance_tracker.sqlite',
  sqliteSeedDemoData: (process.env.SQLITE_SEED_DEMO ?? 'true') === 'true',
  apiBaseUrl:
    process.env.API_BASE_URL ??
    Platform.select({
      android: 'http://192.168.100.7:5001',
      ios: 'http://localhost:5001',
      default: 'http://localhost:5001',
    }),
};
```

URL for emulator and physical android device (Different IP address)

```
apiBaseUrl:
  process.env.API_BASE_URL ??
  Platform.select({
    android: 'http://10.0.2.2:5001',
    ios: 'http://localhost:5001',
    default: 'http://localhost:5001',
  }),
```

```
apiBaseUrl:
  process.env.API_BASE_URL ??
  Platform.select({
    android: 'http://192.168.100.7:5001',
    ios: 'http://192.168.0.6:5001',
    default: 'http://localhost:5001',
  }),
```

Authentication Header Management

Every REST API call includes Bearer token authentication:

- Retrieves JWT token from session (stored after login)
- Throws error if user not authenticated
- Used in all API calls to verify user identity
- Server validates token and returns 401 if invalid

File: services/transactionApi.ts

```
function getAuthHeaders(): Record<string, string> {
  const session = getAuthSession();
  if (!session?.token) {
    throw new Error('Please log in first');
  }

  return {
    Authorization: `Bearer ${session.token}`,
  };
}
```

Response Parsing with Error Handling

Error Handling Strategy:

- Checks HTTP status code
- Extracts error message from JSON payload
- Attaches status code for specific handling (e.g., 401 auth errors)
- Handles non-JSON responses gracefully

File: services/transactionApi.ts

```
async function parseResponse<T>(response: Response): Promise<T> {
  const contentType = response.headers.get('content-type') ?? '';
  const isJson = contentType.toLowerCase().includes('application/json');
  const payload = isJson ? await response.json() : null;

  if (!response.ok) {
    const message = payload?.message ?? `Request failed (${response.status})`;
    const err: any = new Error(message);
    // attach status so callers can react to auth errors (401)
    err.status = response.status;
    err.payload = payload;
    throw err;
  }

  return payload as T;
}
```

Network Error Detection:

- Distinguishes between network failures and application errors to implement appropriate retry logic.

File: services/transactionApi.ts

```
function isNetworkError(error: unknown): boolean {
  const message = String((error as any)?.message ?? '').toLowerCase();
  return message.includes('network request failed') || message.includes('failed to fetch');
}
```

Transaction Synchronization Flow

Phase 1: Local Creation

When a user creates a transaction while offline:

1. Get Session: Retrieves authenticated user's ID
2. Insert Locally: Stores transaction in SQLite with temporary ID
3. Queue Change: Records operation in pending changes table
4. Update UI: Updates pending count badge
5. Trigger Sync: Initiates sync process (if online)
6. Return Immediately: User sees transaction instantly (optimistic update)

File: services/transactionApi.ts

```
export async function insertTransaction(input: CreateTransactionInput): Promise<Transaction> {
  const session = getAuthSession();
  const localTransaction = await insertTransactionLocal({
    ...input,
    userId: session?.userId ?? input.userId,
  });

  await addPendingTransactionChange('create', localTransaction.id, localTransaction);
  setPendingCount((await getPendingTransactionChanges()).length);
  clearTransactionCache();
  void syncPendingTransactions();
  return localTransaction;
}
```

Phase 2: Continuous Sync Monitoring

The application runs a background sync every 15 seconds:

Sync Strategy:

- **Sequential Processing:** Handles changes one at a time in order
- **FIFO Order:** Processes oldest changes first (maintains causality)
- **Atomic Operations:** Each change completes fully before moving to next
- **Error Tolerance:** Stops on network errors (retries later), also stops on server errors

File: services/transactionApi.ts

```

export async function syncPendingTransactions(): Promise<void> {
  if (isSyncInProgress) {
    return; // Prevent concurrent sync attempts
  }

  const session = getAuthSession();
  if (!session?.token) {
    return; // No session, can't sync
  }

  isSyncInProgress = true;
  setSyncing(true);
  try {
    const pending = await getPendingTransactionChanges();
    setPendingCount(pending.length);

    // Process each pending change in order
    for (const change of pending) {
      const payload = JSON.parse(change.payload || '{}');

      try {
        if (change.operationType === 'create') {
          // Handle CREATE operation
        } else if (change.operationType === 'update') {
          // Handle UPDATE operation
        } else if (change.operationType === 'delete') {
          // Handle DELETE operation
        }
      } catch (error) {
        if (isNetworkError(error)) {
          break; // Network issue, stop and retry later
        }
        break; // Server error, stop to preserve order
      }
    }
  } finally {
    isSyncInProgress = false;
    setSyncing(false);
    const remaining = (await getPendingTransactionChanges()).length;
    setPendingCount(remaining);
  }
}

```

Phase 3: CREATE Operation Detail

1. POST to `/api/transactions` with transaction details'
2. Server generates permanent ID and stores in database
3. Client receives ID and updates all local references
4. Removes operation from pending queue
5. Continues to next pending operation

File: `services/transactionApi.ts` (inside `syncPendingTransactions()`)

```

if (change.operationType === 'create') {
  const response = await fetch(`${config.apiUrl}/api/transactions`, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      ...getAuthHeaders(),
    },
    body: JSON.stringify({
      amount: Number(payload.amount),
      type: payload.type,
      category: payload.category,
      date: payload.date,
      note: payload.note ?? '',
      receiptUrl: payload.receiptUrl ?? '',
      walletId: payload.walletId,
      userId: payload.userId ?? session.userId,
    })),
  });
  const created = await parseResponse<{ id: string | number }>(response);
  const newId = String(created.id);
  if (newId && newId !== change.transactionId) {
    await replaceTransactionIdReferences(change.transactionId, newId);
  }
  await removePendingTransactionChange(change.id);
  continue;
}

```

Phase 4: UPDATE and DELETE Operations

- UPDATE: Uses PUT method, only sends changed fields
- DELETE: Uses DELETE method, no body required
- Both receive confirmation and remove from pending queue

File: `services/transactionApi.ts` (inside `syncPendingTransactions()`)

UPDATE:

```

if (change.operationType === 'update') {
  const response = await fetch(`${config.apiUrl}/api/transactions/${encodeURIComponent(change.transactionId)}`, {
    method: 'PUT',
    headers: {
      'Content-Type': 'application/json',
      ...getAuthHeaders(),
    },
    body: JSON.stringify({
      amount: Number(payload.amount),
      type: payload.type,
      category: payload.category,
      date: payload.date,
      note: payload.note ?? '',
      receiptUrl: payload.receiptUrl ?? '',
      walletId: payload.walletId,
    }),
  });

  await parseResponse({ affected: number }(response);
  await removePendingTransactionChange(change.id);
  continue;
}

```

DELETE:

```

if (change.operationType === 'delete') {
  const response = await fetch(`${config.apiUrl}/api/transactions/${encodeURIComponent(change.transactionId)}`, {
    method: 'DELETE',
    headers: {
      ...getAuthHeaders(),
    },
  });

  await parseResponse({ affected: number }(response);
  await removePendingTransactionChange(change.id);
}

```

Phase 5: Sync Timer Management

- Starts immediately when called
- Runs every 15 seconds (15000 ms)
- Can be stopped to prevent unnecessary network activity
- Typically started on app launch and stopped on logout

File: services/transactionApi.ts

```

export function startTransactionSync(): void {
  if (syncTimer) {
    return;
  }

  void syncPendingTransactions();
  syncTimer = setInterval(() => {
    void syncPendingTransactions();
  }, 15000);
}

```

```

export function stopTransactionSync(): void {
  if (!syncTimer) {
    return;
  }

  clearInterval(syncTimer);
  syncTimer = null;
}

```

Data Fetching and Merging

- Start with server data (source of truth for synced items)
- Override with local data (reflects user's latest changes)
- Respect pending deletes (don't show deleted items)
- Sort by date for chronological display

Fetching Transactions with Merge Strategy

The application implements a Local-First Synchronization Pattern, ensuring that users always have access to their financial data regardless of network stability. The *getTransactionsByUser* function serves as the primary engine for this “Source of Truth” reconciliation.

Execution Flow:

1. **Pre-Sync:** Triggers *syncRecurringTransactions()* to auto-generate due transactions locally before cloud communication.
2. **Local Snapshot:** Retrieves current records from the SQLite database and applies wallet-specific filters.
3. **Proactive Sync:** Identifies *pendingChanges* and initiates a background sync to push local updates to the server.
4. **Cloud Retrieval:** Issues a RESTful GET request to */api/transactions* using JWT Bearer Token authentication
5. **Smart Merge:** Resolves data conflicts by prioritizing local pending changes over server records.
6. **Offline Fallback:** Returns the local dataset if the network request fails, ensuring zero downtime.

File: *services/transactionApi.ts*

```
export async function getTransactionsByUser(_userId?: string, walletId?: string): Promise<Transaction[]> {
  try {
    await syncRecurringTransactions();
    clearTransactionCache();
  } catch (err) {
    console.warn('Recurring transaction sync skipped while offline.');
```

```
  }

  const fallbackUserId = getUserIdForFallback(_userId);
  const localRows = fallbackUserId ? await getTransactionsByUserLocal(fallbackUserId) : await getAllTransactions();
  const localData = walletId
    ? localRows.filter(item => String(item.walletId ?? '') === String(walletId))
    : localRows;

  const pendingChanges = await getPendingTransactionChanges();

  await syncPendingTransactions();

  let url = `${config.apiUrl}/api/transactions`;
  if (walletId) {
    url += `?walletId=${encodeURIComponent(walletId)}`;
  }
}
```

```

try {
  const response = await fetch(url, {
    headers: {
      ...getAuthHeaders(),
    },
  });
}

const rows = await parseResponse<any[]>(response);
const serverData = rows.map(mapTransactionRow);
const filteredServerData = walletId
  ? serverData.filter(item => String(item.walletId ?? '') === String(walletId))
  : serverData;

return mergeLocalAndServerTransactions(localData, filteredServerData, pendingChanges);
} catch (error) {
  return localData;
}
}

```

The Merge Algorithm

The *mergeLocalAndServerTransactions* function is the core logic for resolving data conflicts between the Cloud Server and Local SQLite database. It ensures the UI reflects the most current user state, even before background synchronization completes.

Algorithm Implementation Steps:

1. **Identify Pending Deletions:** The algorithm creates a Set of transaction IDs marked for deletion in the pendingChanges queue.
2. **Filtered Cloud Import:** It populates a Map with all records from the server, explicitly skipping any IDs found in the “deleted” set to prevent “ghost” data from reappearing.
3. **Local Priority Override:** It iterates through local SQLite rows and adds them to the Map. Because a Map key is unique, any existing server record with the same ID is overwritten by the local version, ensuring “Optimistic UI” updates take priority.
4. **Chronological Sorting:** The merged values are converted to an array and sorted by date in descending order using *localeCompare* for a consistent history view.

File: services/transactionApi.ts

```
function mergeLocalAndServerTransactions(localRows: Transaction[], serverRows: Transaction[],
    pendingChanges: Array<{ operationType: string; transactionId: string }>): Transaction[] {
    const merged = new Map<string, Transaction>();
    const deletedIds = new Set(
        pendingChanges
            .filter(change => change.operationType === 'delete')
            .map(change => String(change.transactionId)),
    );

    for (const row of serverRows) {
        if (!deletedIds.has(String(row.id))) {
            merged.set(String(row.id), row);
        }
    }

    for (const row of localRows) {
        merged.set(String(row.id), row);
    }

    return Array.from(merged.values()).sort((left, right) => right.date.localeCompare(left.date));
}
```

C.2.2 WebSocket Implementation

Socket Service Architecture

The *socketService.ts* implements a singleton pattern for managing WebSocket connections:

Design Pattern:

- Singleton: Only one instance manages all WebSocket operations
- Listener Pattern: Decoupled observers receive events
- Alert Queuing: Stores missed alerts until listeners subscribe

File: *services/socketService.ts*

```
class SocketService {
  private socket: any | null = null;
  private budgetAlertListeners: Set<NotificationListener> = new Set();
  private recurringListeners: Set<NotificationListener> = new Set();
  private connectListeners: Set<() => void> = new Set();
  private disconnectListeners: Set<() => void> = new Set();
  private missedBudgetAlerts: BudgetAlert[] = [];

  export default new SocketService();
```

Socket Client Lazy Loading

To support offline scenarios where *socket.io-client* might not be available.

Benefit: Application gracefully degrades if WebSocket client library is unavailable.

File: *services/socketService.ts*

```
function loadSocketClient(): SocketClientModule | null {
  if (socketClientModule) {
    return socketClientModule;
  }

  try {
    // eslint-disable-next-line @typescript-eslint/no-var-requires
    socketClientModule = require('socket.io-client');
    return socketClientModule;
  } catch (error) {
    console.warn('[Socket] socket.io-client is unavailable. Running without live socket features.', error);
    socketClientModule = null;
    return null;
  }
}
```

Connection Establishment

Connection Configuration:

- Namespace: Uses /finance namespace for organization
- Transport: WebSocket only (no fallback to polling)

- Reconnection: Auto-reconnects with exponential backoff
- Initial delay: 1000ms
- Max delay: 5000ms
- Max attempts: 5
- forceNew: Creates new connection (prevents caching issues)

File: services/socketService.ts

```
connect(serverUrl: string = config.apiBaseUrl): Promise<void> {
  return new Promise<void>((resolve, reject) => {
    try {
      const socketClient = loadSocketClient();
      if (!socketClient?.io) {
        this.socket = null;
        resolve();
        return;
      }

      const normalizedUrl = String(serverUrl || '').trim();
      const baseUrl = normalizedUrl ? normalizedUrl.replace(/\/api\/?$/, '') : 'http://localhost:5001';

      this.socket = socketClient.io(`${baseUrl}/finance`, {
        transports: ['websocket'],
        reconnection: true,
        reconnectionDelay: 1000,
        reconnectionDelayMax: 5000,
        reconnectionAttempts: 5,
        forceNew: true,
      });
    }
  });
}
```

Connection Event Handling

Connection Flow:

1. WebSocket connects to server
2. Client emits *user_login* event with user ID
3. Server can now track which user this connection belongs to
4. Connection listeners are triggered
5. Promise resolves to indicate ready state

File: services/socketService.ts

```
this.socket.on('connect', () => {
  console.log('[Socket] Connected to server');

  // Send user login event with userId
  const session = getAuthSession();
  if (session?.userId) {
    this.socket?.emit('user_login', { userId: session.userId });
    console.log('[Socket] Emitted user_login for userId: ${session.userId}');
  }

  this.connectListeners.forEach(listener => listener());
  resolve();
});

this.socket.on('error', (error) => {
  console.error('[Socket] Connection error:', error);
  reject(error);
});
```

Output:

```
[Socket] User connected: tsnnQ9mDtINQ3qH-AAAD
[Socket] User 3 joined room
```

Listening for Budget Alerts

Alert Queuing Logic:

- If no listeners subscribed yet, alert is queued
- When first listener subscribes, all queued alerts are flushed
- This prevents alert loss during app initialization

File: services/socketService.ts

```
// Listen for budget alerts
this.socket.on('budget_alert', (data: BudgetAlert) => {
  console.log('[Socket] Budget alert received:', data);
  if (this.budgetAlertListeners.size === 0) {
    this.missedBudgetAlerts.push(data);
  } else {
    this.budgetAlertListeners.forEach(listener => listener(data));
  }
});
```

Budget Verification Interface

When a transaction is added, the client can proactively check budget:

Usage Flow:

1. User adds transaction to Food category
2. Client calculates new total in that category
3. Compares against budget limit
4. Emits *check_budget* event to server
5. Server evaluates and may respond with *budget_alert*

File: services/socketService.ts

```
// Send budget check to server
checkBudget(userId: string, category: string, spent: number, limit: number): void {
  if (this.socket) {
    this.socket.emit('check_budget', {
      userId,
      category,
      spent,
      limit,
    });
    console.log(`[Socket] Sent check_budget for ${category}`);
  } else {
    console.warn('[Socket] Not connected. Cannot send check_budget');
  }
}
```

Recurring Sync Notifications

Integration with Backend:

- Called after background task syncs recurring transactions
- Server may respond with recurring_synced notification
- Notifies user of automatic transaction generation

File: services/socketService.ts

```
// Send recurring sync notification
syncRecurring(userId: string, syncedCount: number): void {
  if (this.socket) {
    this.socket.emit('sync_recurring', {
      userId,
      syncedCount,
    });
    console.log(`[Socket] Sent sync_recurring: ${syncedCount} transactions`);
  } else {
    console.warn(`[Socket] Not connected. Cannot send sync_recurring`);
  }
}
```

Subscription Management for Budget Alerts

- Returns unsubscribe function (for cleanup in React useEffect)
- Flushes queued alerts when first listener connects
- Multiple listeners can subscribe (fan-out pattern)

File: services/socketService.ts

```
// Subscribe to recurring sync notifications
subscribeToRecurringSync(listener: NotificationListener): () => void {
  this.recurringListeners.add(listener);
  console.log(`[Socket] Recurring sync listener added`);

  // Return unsubscribe function
  return () => {
    this.recurringListeners.delete(listener);
    console.log(`[Socket] Recurring sync listener removed`);
  };
}
```

C.2.3 Currency Exchange Rate Integration

Content

```
C:\Users\Kelly\WAD_Assignment\Personal-Finance-Tracker\db>node service.js

🚀 Starting backend...
✓ Database tables created
✓ Database seeded
Server running on port 5001
Database: C:\Users\Kelly\WAD_Assignment\Personal-Finance-Tracker\db\finance_tracker.sqlite
✓ WebSocket listening on /finance namespace
✓ Backend ready for connections
[Socket] User connected: K2I9Gu1QLr3_-nN4AAAB
[Socket] User 3 joined room
[2026-05-02T03:30:43.458Z] POST /api/recurring-transactions/sync
[2026-05-02T03:30:43.465Z] GET /api/wallets
[2026-05-02T03:30:43.468Z] POST /api/recurring-transactions/sync
[2026-05-02T03:30:43.470Z] GET /api/wallets
[2026-05-02T03:30:43.471Z] GET /api/wallets
[2026-05-02T03:30:43.721Z] GET /api/transactions
[2026-05-02T03:30:43.723Z] GET /api/transactions
[2026-05-02T03:30:43.748Z] POST /api/recurring-transactions/sync
```

REFERENCES

- React Native (2024). *React Native · A framework for building native apps using React*. [online] reactnative.dev. Available at: <https://reactnative.dev/>. [Accessed 15 Apr. 2026]
- Alpha Vantage (2017). *API Documentation | Alpha Vantage*. [online] Alphavantage.co. Available at: <https://www.alphavantage.co/documentation> [Accessed 20 April 2026].
- MDN Web Docs. (n.d.). *Using Fetch*. [online] Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch [Accessed 23 Apr. 2026].
- Reactnative.dev. (2025). *React Native · Learn once, write anywhere*. [online] Available at: <https://reactnative.dev/docs/0.81/asyncstorage> [Accessed 19 Apr. 2026].
- Expo Documentation. (n.d.). *AsyncStorage*. [online] Available at: <https://docs.expo.dev/versions/latest/sdk/async-storage/> [Accessed 15 Apr. 2026].

END OF DOCUMENT